

System Components

Data Classification

Data Type	Classification	Storage Location	Controls
JWT	Confidential	Browser local storage / request headers	Expiry, signature verification, bearer auth.
Tenant ID	Internal	PostgreSQL, JWT claims	Tenant isolation, RLS.
User Metadata	Internal	PostgreSQL users table	Tenant-scoped access.
Workflow Definitions	Internal	PostgreSQL workflows table	Tenant isolation, authenticated access.
Node Configuration	Internal	PostgreSQL workflow definition JSON	Tenant isolation.
Credentials	Restricted	Credential Vault	AES encryption, no plaintext display, rotation support.
OAuth Tokens	Restricted	OAuth connections table	Encrypted storage, scoped access.
Execution Logs	Internal	PostgreSQL execution_logs table	TTL retention, tenant isolation.
Audit Logs	Internal / Compliance Evidence	PostgreSQL audit_log + WORM archive	Hash chaining, archival, restricted access.
External Connector Payloads	Confidential	Transient runtime processing	Connector-specific authorization and minimal retention.

Data Flow Details

1. Authentication Flow

sequenceDiagram

participant User as Operator

participant UI as Web UI

participant API as API Gateway

User->>UI: Enter login details

UI->>API: POST /auth/dev-login

API->>API: Sign JWT

API-->>UI: accessToken

UI->>API: Authenticated requests with Bearer token

API->>API: Verify JWT and extract tenantId/userId/role

Security controls:

- JWT signature verification.
 - Tenant ID extracted from token.
 - Role included in token payload.
 - Protected routes require `tenantMiddleware`.
-

2. Workflow Builder Flow

sequenceDiagram

participant UI as Web UI

participant API as API Gateway

participant DB as PostgreSQL

UI->>API: POST /workflows

API->>API: Validate JWT and tenant

API->>DB: INSERT workflow with tenant_id

API->>DB: INSERT audit_log workflow.created

DB-->>API: Created workflow

API-->>UI: Workflow saved

Data stored:

- Workflow name.
- Workflow definition JSON.
- Nodes and edges.
- Created by user ID.
- Tenant ID.

Controls:

- Tenant-scoped inserts.
 - PostgreSQL RLS.
 - Audit log event creation.
-

3. Workflow Execution Flow

sequenceDiagram

participant UI as Web UI / Postman
participant API as API Gateway
participant Queue as Redis / BullMQ
participant Worker as Task Orchestrator
participant DB as PostgreSQL
participant SaaS as External Connector

UI->>API: POST /workflows/:id/run
API->>Queue: Enqueue workflow execution job
Queue-->>API: Job ID
API-->>UI: Execution queued
Queue->>Worker: Deliver job
Worker->>DB: Read workflow
Worker->>SaaS: Execute connector action
Worker->>DB: Insert execution_logs success/failed
Worker->>DB: Insert audit_log workflow.executed

Controls:

- Queue-based execution.
 - Execution logs persisted.
 - Completed timestamp recorded.
 - TTL delete-after set.
 - Audit trail written.
-

4. Credential Management Flow

sequenceDiagram

participant Admin as Tenant Admin
participant UI as Web UI
participant API as API Gateway
participant Vault as Credential Vault

participant DB as PostgreSQL

Admin->>UI: Add or rotate credential

UI->>API: Submit secret over authenticated request

API->>Vault: Encrypt secret

Vault->>DB: Store encrypted_payload + iv

DB-->>API: Credential metadata

API-->>UI: Success response without plaintext secret

Controls:

- Plaintext secret accepted only during create/rotate.
 - Secret never displayed in UI.
 - Encrypted payload stored.
 - Credential rotation audit supported.
 - Tenant-scoped credential access.
-

5. Execution History Flow

sequenceDiagram

participant UI as Web UI

participant API as API Gateway

participant DB as PostgreSQL

UI->>API: GET /executions

API->>DB: SELECT tenant-scoped execution_logs

DB-->>API: Execution rows

API-->>UI: Execution history

UI->>API: GET /executions/:id

API->>DB: SELECT execution details

DB-->>API: Execution details

API-->>UI: Details panel

Controls:

- Tenant-scoped read.
 - No cross-tenant access.
 - Status, timestamps, error detail, and TTL are visible to authorized users.
-

6. Tenant Administration Flow

sequenceDiagram

participant Admin as Tenant Admin

participant UI as Web UI

participant API as API Gateway

participant DB as PostgreSQL

Admin->>UI: Open Administration

UI->>API: GET /admin/tenant

API->>DB: Read tenant, members, TTL settings

DB-->>API: Admin data

API-->>UI: Tenant overview

Admin->>UI: Update TTL settings

UI->>API: PATCH /admin/ttl-settings

API->>DB: Upsert tenant_settings

DB-->>API: Updated TTL settings

API-->>UI: Success message

Controls:

- Admin area protected by JWT.
 - Tenant settings stored in PostgreSQL.
 - TTL values persist after refresh.
-

7. Audit Hash Chain Flow

flowchart LR

A[Audit Event 1] -->|current_hash becomes previous_hash| B[Audit Event 2]

B -->|current_hash becomes previous_hash| C[Audit Event 3]

C --> Archive[WORM Archive JSONL + SHA256]

Controls:

- **previous_hash** links audit rows.
 - **current_hash** is computed using event data.
 - Backfill script ensures existing audit rows have hashes.
 - External archive generates SHA256 file hash.
-

8. GDPR Erasure Flow

flowchart TD

```
Request[Right to erasure request] --> Function[right_to_erasure tenant_id]
Function --> OAuth[Delete OAuth connections]
Function --> Settings[Delete tenant settings]
Function --> Vault[Delete credential vault data]
Function --> Executions[Delete execution logs]
Function --> Webhooks[Delete webhook endpoints]
Function --> Audit[Delete audit logs]
Function --> Workflows[Delete workflows]
Function --> Users[Delete users]
Function --> Tenant[Delete tenant]
Tenant --> Verify[Verification count = 0]
```

Controls:

- Erasure is tenant-scoped.
- Deletes dependent records before tenant record.
- Test script verifies deletion.
- Production use should require admin approval and audit evidence before execution.

Storage and Retention

Data Store	Data Held	Retention
PostgreSQL tenants	Tenant metadata	Until tenant deletion or erasure.
PostgreSQL users	Tenant users and roles	Until tenant deletion or erasure.
PostgreSQL workflows	Workflow definitions	Until deletion or tenant erasure.
PostgreSQL execution_logs	Execution history	TTL-based retention.
PostgreSQL credential_vault	Encrypted credential payloads	Until credential deletion, rotation, or tenant erasure.
PostgreSQL audit_log	Audit trail	Long-term retention and external archival.
Redis / BullMQ	Queue jobs	Short-lived execution processing.

WORM Archive

Archived audit JSONL and
SHA256 files

Immutable-style long-term evidence
archive.

Security Controls Summary

Control	Implementation
Authentication	JWT-based access tokens.
Authorization	Role included in JWT and tenant middleware.
Tenant Isolation	Tenant ID enforced in API and PostgreSQL RLS.
Credential Protection	AES encryption, IV storage, no plaintext display.
Audit Integrity	Hash chaining with previous/current hash.
Data Retention	TTL pruning for expired execution logs.
GDPR Erasure	<code>right_to_erasure(tenant_id)</code> utility.
External Audit Evidence	WORM-style JSONL archive with SHA256 hash.
Queue Isolation	Workflow execution jobs include tenant ID.
Operational Visibility	Execution history and status tracking.

Compliance Notes

FlexiBoost supports the following compliance-oriented capabilities:

- Tenant-scoped data isolation.
 - Encrypted credential storage.
 - No plaintext secret exposure in the UI.
 - Execution history and operational traceability.
 - Audit log hash chaining for tamper evidence.
 - External archive generation for audit evidence.
 - Right-to-erasure utility for GDPR workflows.
 - TTL-based retention enforcement.
-

Client Onboarding Data Flow Summary

During client onboarding:

1. A tenant is created.
 2. Tenant administrators sign in.
 3. Credentials are added to the encrypted vault.
 4. Operators build workflows in the visual builder.
 5. Workflow executions are queued through Redis/BullMQ.
 6. The worker processes jobs and invokes external connectors.
 7. Execution logs are stored for monitoring.
 8. Audit events are hash chained and archived.
 9. TTL and erasure controls enforce retention and deletion obligations.
-

Diagram Scope

This document covers application-level data flows for:

- Workflow management.
- Credential management.
- Workflow execution.
- Tenant administration.
- Audit logging.
- WORM archive.
- GDPR erasure.
- TTL pruning.

Infrastructure-level diagrams such as Kubernetes networking, cloud IAM, VPC layout, and production secrets injection are documented separately in production deployment runbooks.